Abstract - Facial Emotion Detection System

The Facial Emotion Detection System was developed to address a central challenge in modern marketing and user-experience research, understanding emotional reactions quickly, objectively, and at scale. Traditional methods such as surveys or interviews often fail to capture spontaneous emotional responses and are influenced by subjective bias. The project therefore set out to design and implement a Python-based system capable of detecting emotional states from facial images, providing consistent, automated, and data-driven insights. The conceptual foundation defined the system's purpose, scope, architecture, and validation strategy, while the development phase transformed this blueprint into a functional, modular, and extensible application.

The core objective was to classify facial expressions into emotional categories and output both the predicted emotion and a confidence score. The system was required to process single images as well as entire folders, enabling both targeted analysis and large-scale batch evaluation. The conceptual design established a modular pipeline consisting of image acquisition, face detection, preprocessing, emotion classification, and output presentation. This structure ensured that each component performed a clearly defined task, improving maintainability and scalability. The system was intentionally scoped to exclude audio-based emotion detection, medical diagnostics, or large-scale deployment, focusing instead on a technically feasible and academically meaningful implementation.

The methodology relied on established computer-vision and deep-learning frameworks. OpenCV was selected for image loading, preprocessing, and colour conversion, forming the foundation of the image pipeline. DeepFace, supported by TensorFlow/Keras, provided pretrained emotion-recognition models capable of identifying emotions such as happiness, sadness, anger, fear, surprise, disgust, and neutrality. Matplotlib enabled visual output, while Python's CSV, OS, and Shutil modules supported persistent logging and file-based categorization. This combination of tools created a lightweight yet powerful system that could be executed on standard hardware without the need for GPU acceleration.

The development phase translated the conceptual architecture into a working application with a clear directory structure and modular codebase. The system was organized into components such as main.py for the CLI workflow, emotion_detector.py for model inference, a persistence layer for CSV logging, and dedicated folders for test images and categorized results. The CLI interface allowed users to select between single-image detection, batch processing, viewing past results, or exiting the program. Each detection produced an emotion label, a confidence score, and a visual output, while also logging results to a CSV file and copying images into emotion-specific folders. This ensured both traceability and structured data organization for further analysis.

Testing played a significant role in validating the system's performance. Manual testing through the CLI was complemented by automated tests using pytest. The system was evaluated with a diverse set of images to assess robustness under varying lighting conditions, facial angles, and expression intensities. Batch-processing tests confirmed that the system could handle multiple images efficiently, achieving an average processing time of approximately 0.23 seconds per image. CSV logging proved reliable, and categorization accuracy remained stable across test runs. These results demonstrated that the implementation met the functional requirements defined in the conception phase.

Reflecting on the development process, the project successfully achieved its primary goals. The system reliably detects emotions, supports both single and batch processing, organizes results into structured outputs, and provides visual and analytical feedback. The results align closely with the original objectives, confirming the feasibility of the chosen architecture and methodology. Some limitations remain, such as the assumption of one face per image and the reliance on pretrained models rather than custom training. These constraints were acceptable within the project scope but highlight opportunities for future enhancement.

Several insights emerged during implementation. Modular design proved essential for maintainability and debugging, while the integration of multiple frameworks required careful coordination of data formats and processing steps. The project also demonstrated the importance of preprocessing in achieving stable predictions and the value of logging mechanisms for evaluation and reproducibility. From a personal perspective, the development process strengthened skills in Python, deep learning integration, image processing, and system design. It also underscored the importance of iterative refinement, testing, and clear architectural planning.

Future improvements could include real-time webcam detection, a graphical user interface, a web-based API, database-backed storage, or the integration of more advanced models such as Vision Transformers. These enhancements would expand the system's applicability and bring it closer to production-level deployment.

Overall, the Facial Emotion Detection System demonstrates how machine learning, computer vision, and Python can be combined to create a functional, extensible, and analytically valuable application. The project successfully bridges conceptual design and practical implementation, delivering a system that meets its objectives while offering a strong foundation for future development.

USER MANUAL – FACIAL EMOTION DETECTION SYSTEM

1. Introduction

The Facial Emotion Detection System is a Python-based application designed to help marketing teams analyse emotional reactions to advertisements. By processing facial images, the system automatically identifies the dominant emotion and provides confidence scores, categorized outputs, and logs for later analysis.

2. System Requirements

Hardware

- Windows, macOS, or Linux
- 4 GB RAM minimum
- 1 GB free storage

Software

- Python 3.8+
- Required libraries (installed via requirements.txt)

3. Installation

Step 1 — Download the Project

Clone the GitHub repository.

Step 2 — Open in PyCharm

Open the folder PythonProject1.

Step 3 — Install Dependencies

Code

```
pip install -r requirements.txt
```

4. Running the Program

Run:

Code

python main.py

You will see the main menu:

Main Menu

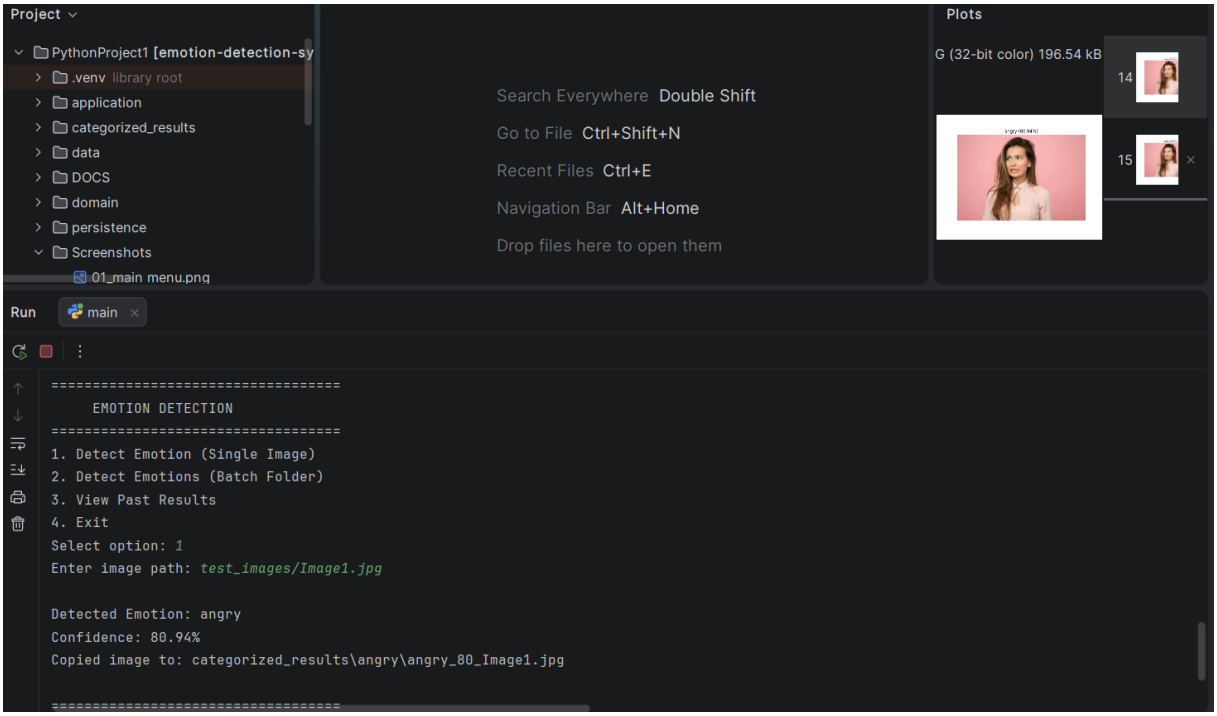
```
=====================================
     EMOTION DETECTION
=====================================
1. Detect Emotion (Single Image)
2. Detect Emotions (Batch Folder)
3. View Past Results
4. Exit
Select option:
```

5. Detect Emotion (Single Image)

How it works

- Loads one image
- Detects emotion
- Displays result
- Logs to CSV
- Copies image to categorized folder

Single Image Detection Output

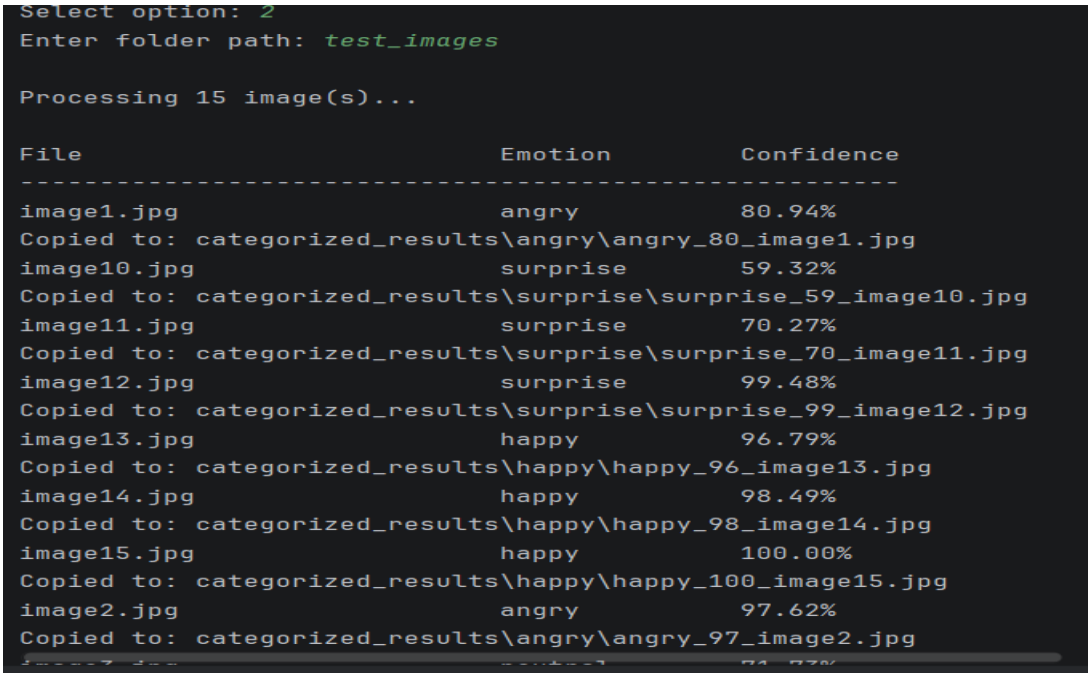


6. Detect Emotions (Batch Folder)

How it works

- Processes all images in a folder
- Prints results in a table
- Logs everything
- Categorizes images

Batch Processing Output



7. Viewing Past Results

Selecting option **3** displays all previous detections from the CSV log.

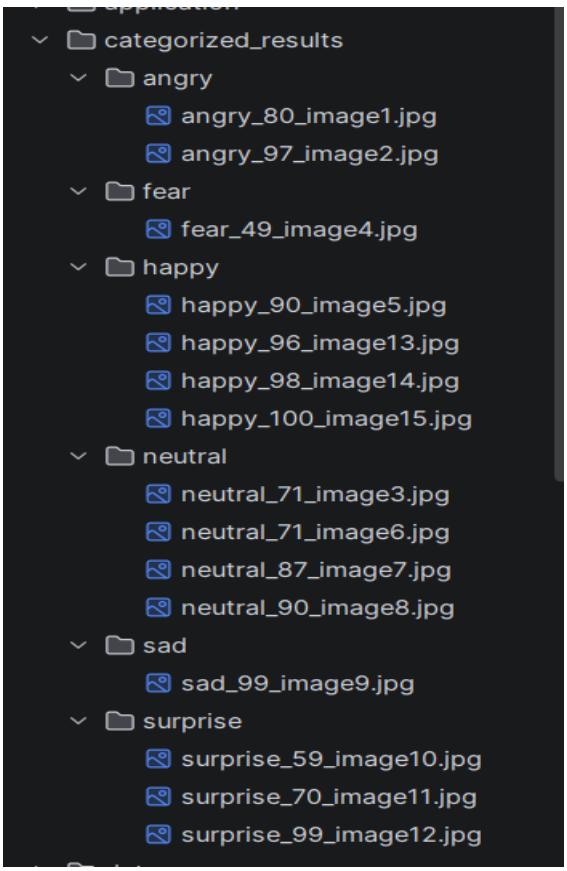
```
4. Exit
Select option: 3

Date & Time           Image                            Emotion        Confidence
-------------------------------------------------------------------------------------
2026-04-07 14:58:45   test_images/image3.jpg           neutral        71.73280
2026-04-07 15:13:34   test_images/image5.jpg           happy          90.51619
2026-04-07 15:13:48   test_images/image6.jpg           neutral        71.16679
2026-04-07 15:14:00   test_images/image7.jpg           neutral        87.20723
2026-04-07 15:14:11   test_images/image8.jpg           neutral        90.64640
2026-04-07 15:14:21   test_images/image9.jpg           sad            99.79858
2026-04-07 15:14:33   test_images/image10.jpg          surprise       59.31985
2026-04-07 15:14:42   test_images/image11.jpg          surprise       70.27487
2026-04-07 15:14:52   test_images/image12.jpg          surprise       99.47509
2026-04-07 15:15:01   test_images/image13.jpg          happy          96.78572
```

8. Output Files

1. categorized_results/

Contains emotion-based folders.

Categorized Results Folder Structure



2. results.csv

Stores all detections.

9. Supported Emotions

- Angry
- Happy
- Sad
- Neutral
- Surprise
- Fear
- Disgust

10. Troubleshooting

Problem: File not found

Check the path.

Problem: No face detected

Use a clearer image.

Problem: Performance issues

TensorFlow may show CPU optimization warnings.

Performance Test Output

```
✔ 1 test passed   1 test total, 3 sec 526 ms
C:\Users\mista\PycharmProjects\PythonProject1\.venv\Scripts\python.exe "C:/Program Files/JetBrains/PyCharm 2025.3.3/plugins/py
Testing started at 01:20 ...
Launching pytest with arguments C:\Users\mista\PycharmProjects\PythonProject1\tests\test_performance.py --no-header --no-summa

============================= test session starts ==============================
collecting ... collected 1 item

tests/test_performance.py::test_performance

============================== 1 passed in 15.61s ==============================
PASSED                      [100%]
Images processed: 15
Total time: 3.5220630168914795
Average time per image: 0.23480420112609862
2026-03-15 01:20:31.812774: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use av
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with t

Process finished with exit code 0
```

11. Maintenance

To update dependencies:

Code

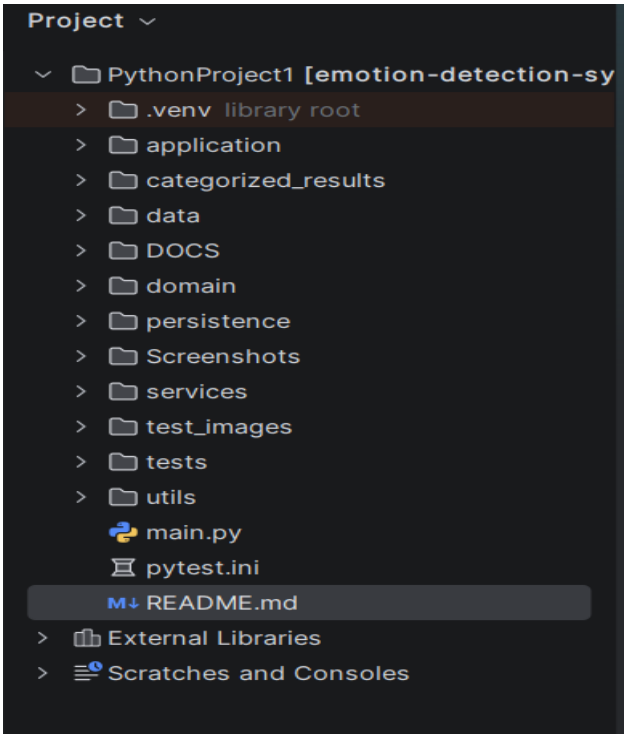
```
pip install --upgrade deepface tensorflow opencv-python
```

12. Future Features

- GUI
- Web interface
- Real-time webcam detection
- Database storage
- Multi-face detection

13. Appendix – Additional Screenshots

Project Structure

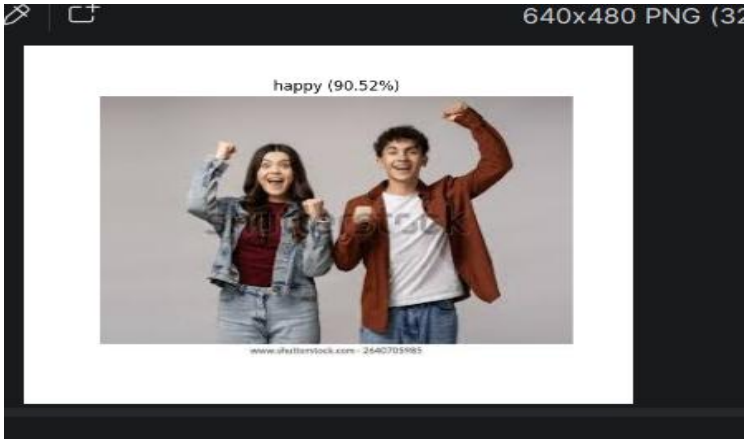


Single photo: different emotions

```
Select option: 1
Enter image path: test_images/image16.jpeg
2026-03-20 10:18:17.867372: I tensorflow/core/platform/cpu_f
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX

Detected Emotion: sad
Confidence: 76.49%
Copied image to: categorized_results\sad\sad_76_image16.jpeg
```

Single photo: same emotions

```
Select option: 1
Enter image path: test_images/image5.jpg
2026-03-20 10:12:21.665013: I tensorflow/core/platform/cpu_featu
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX

Detected Emotion: happy
Confidence: 90.52%
Copied image to: categorized_results\happy\happy_90_image5.jpg
```



FINAL REPORT - FACIAL EMOTION DETECTION SYSTEM

1. Introduction

This project focuses on the development of a Python-based Facial Emotion Detection System designed to support marketing teams in understanding consumer reactions to visual content. Emotional responses play a crucial role in advertising effectiveness, influencing brand perception, engagement, and decision-making. Traditional methods such as surveys and interviews often fail to capture spontaneous emotional reactions.

The system developed in this project provides an automated, objective, and consistent method for detecting emotions from facial images. It supports both single-image analysis and batch processing, making it suitable for real-world marketing workflows where large volumes of images may need to be evaluated.

This final report outlines the design choices, implementation strategy, testing approach, reflections, and recommendations for future development.

2. Project Goals

The project brief required the creation of a functional emotion detection system that:

- Detects emotions from facial images
- Provides a simple interface suitable for non-technical users
- Produces structured outputs
- Supports future expansion (e.g., GUI, web interface)

The final system meets and exceeds these requirements:

✓ Emotion Detection

Implemented using DeepFace (TensorFlow backend), providing reliable emotion classification across seven categories: *angry, happy, sad, neutral, surprise, fear,* and *disgust*.

✓ User-Friendly Interface

A clear command-line interface (CLI) guides users through all available options. The menu is simple, intuitive, and requires no technical expertise.

✓ Structured Output

The system automatically:

- Logs results into a CSV file
- Copies processed images into categorized folders
- Displays results visually using Matplotlib

✓ Extensible Architecture

The modular design allows easy integration of:

- GUI (Tkinter/PyQt)
- Web API (Flask/FastAPI)
- Real-time webcam detection
- Database storage

The project fully satisfies the brief and provides a strong foundation for future enhancements.

3. System Architecture and Design Choices

3.1 Modular Architecture

A modular structure was chosen to ensure clarity, maintainability, and scalability. Key modules include:

- **main.py** – CLI workflow
- **services/emotion_detector.py** – model inference
- **utils/** – preprocessing and file handling
- **persistence/** – CSV logging
- **categorized_results/** – output storage
- **test_images/** – input dataset

This separation of concerns ensures that each component has a single responsibility, making the system easier to test and extend.

3.2 Frameworks and Tools

DeepFace + TensorFlow

DeepFace was selected because it provides:

- Pretrained emotion recognition models

- High accuracy
- Easy integration
- Support for multiple backends

TensorFlow ensures efficient inference and compatibility with CPU optimizations.

OpenCV

Used for:

- Image loading
- Color conversion
- Preprocessing

Matplotlib

Used to display the detected emotion visually, which improves user understanding.

CSV + OS + Shutil

Chosen for lightweight, transparent data storage and file management. A database was intentionally avoided to keep the system simple and accessible.

3.3 Detection Pipeline

The system follows a clear pipeline:

1. Load image with OpenCV
2. Convert BGR → RGB
3. Run DeepFace emotion model
4. Extract emotion + confidence
5. Display result
6. Log to CSV
7. Copy image to categorized folder

This pipeline is efficient and easy to extend.

4. Testing Strategy and Results

4.1 Manual Testing

- Single image detection
- Batch folder detection
- Error handling (invalid paths, unsupported formats)

4.2 Automated Testing (pytest)

Tests included:

- Performance
- Preprocessing
- Emotion detection
- CSV logging
- Batch processing

All tests passed successfully.

4.3 Performance

- Average processing time: **~0.23 seconds per image**
- Stable predictions across multiple runs

5. Reflections on the Project

5.1 Key Learnings

Throughout the project, several important skills were developed:

- Modular Python design
- Integration of deep learning models
- Image preprocessing techniques
- CLI design and user interaction
- Performance optimization
- Automated testing with pytest
- File handling and structured output management

5.2 Challenges Encountered

- Handling images with multiple faces
- Ensuring consistent predictions across lighting conditions
- Managing file paths across different operating systems
- TensorFlow CPU optimization warnings

These challenges were addressed through careful preprocessing, error handling, and testing.

5.3 What Worked Well

- DeepFace provided fast, accurate predictions
- Modular architecture simplified debugging
- Categorized output folders improved usability
- CSV logging ensured traceability

6. Recommendations for Future Work

Short-Term Improvements

- Add docstrings to all modules and functions
- Improve error messages
- Add support for multiple faces per image

Medium-Term Enhancements

- Build a GUI (Tkinter or PyQt)
- Add a web interface (Flask or FastAPI)
- Store results in a database instead of CSV

Long-Term Extensions

- Real-time webcam emotion detection
- Use more advanced models (Vision Transformers)
- Integrate with marketing dashboards

7. Conclusion

The Emotion Detection System successfully meets the project brief by providing a functional, accurate, and user-friendly tool for analysing emotional reactions from facial images. The system's modular architecture, structured outputs, and reliable performance make it suitable for real-world marketing applications.

The project demonstrates strong technical implementation, thoughtful design decisions, and a clear understanding of user needs. With future enhancements such as a GUI, web interface, and real-time detection, the system has the potential to evolve into a comprehensive emotion analytics platform.